

Chapter 8

System Dependability & Critical Systems

BAIT3153 Software and
Project Management (SPM)



System Dependability & Critical Systems

Table of Contents

- 8.1 **Critical Systems**
- 8.2 **Dependability**
- 8.3 Achieving Dependability
- 8.4 Availability & Reliability
- 8.5 Safety
- 8.6 Security



8.1 Critical Systems

What is a **critical system**?

An event that occurs at some point in time when *the system **does not** deliver a service as expected by its users*

- A system whose **failures** can result in:
 - Significant economic losses,
 - Physical damage or
 - Threats to human life.
- There are **3 types of critical systems**:
 - a. **Safety-critical** systems
 - b. **Mission-critical** systems
 - c. **Business-critical** systems.

at some point in time *the system **does not** deliver a service as expected by its users*

8.1 Critical Systems

There are 3 types of **critical systems**:

a. Safety-Critical Systems

- A **system failure** may cause:
 - injury,
 - loss of life or
 - major environmental damage.
- **Examples:**
 - **Self-driving Cars**
(failure of the braking system may cause)
 - **Emergency Lighting System**
(system failure could lead to injuries during an evacuation)
 - Flight auto-pilot control system
 - _____

b. Mission-Critical Systems

- A **system failure** may result in the failure of some goal-directed activity.
- **Examples:**
 - **GPS** (Using GPS, the goal is ...)
(If GPS failed...)
 - **Attendance system**
(The goal of attendance_sys is ...)
(If attendance_sys failed ...)
 - _____

c. Business-Critical Systems

- A **systems failure** can have serious consequences for the business
- **Examples:**
 - **ERP System**
(ERP system failure can result in **serious consequences** for the business)
 - **Online banking system**
(OBS failure also can have serious consequences for the business)
 - **E-commerce website**
 - _____

Car plate recognition system is a Mission-Critical or Business-Critical sys? ____



8.1 Critical Systems

Causes of System Failure:

1. Failure of Hardware – due to *design and manufacturing errors* or components have reached their *end of life*.
2. Failure of Software - due to *errors in code*.
3. Failure of Operation – **due to Human errors**, perhaps the largest single cause of system failures in socio-technical systems.

E.g during system operation, human operator:

- i. accidentally deleted some important system files
(e.g: .ini files, .exe files, .sys files, etc
- i. not configuring a system correctly

Costs of System Failure

- Direct failure costs – **system need to be replaced.**
- Indirect failure costs
 - lost of revenue & reputation
 - litigation (lawsuits) costs
 - etc

For **critical system** the costs of failure is often very high **—**



Table of Contents

8.1 Critical Systems

- A system whose failures can result in
 - Significant economic losses,
 - Physical damage or
 - Threats to human life.
- 3 types of critical systems
- Causes of System Failure: ...
- Costs of Failure: ...

8.2 Dependability

8.3 Achieving Dependability

8.4 Availability & Reliability

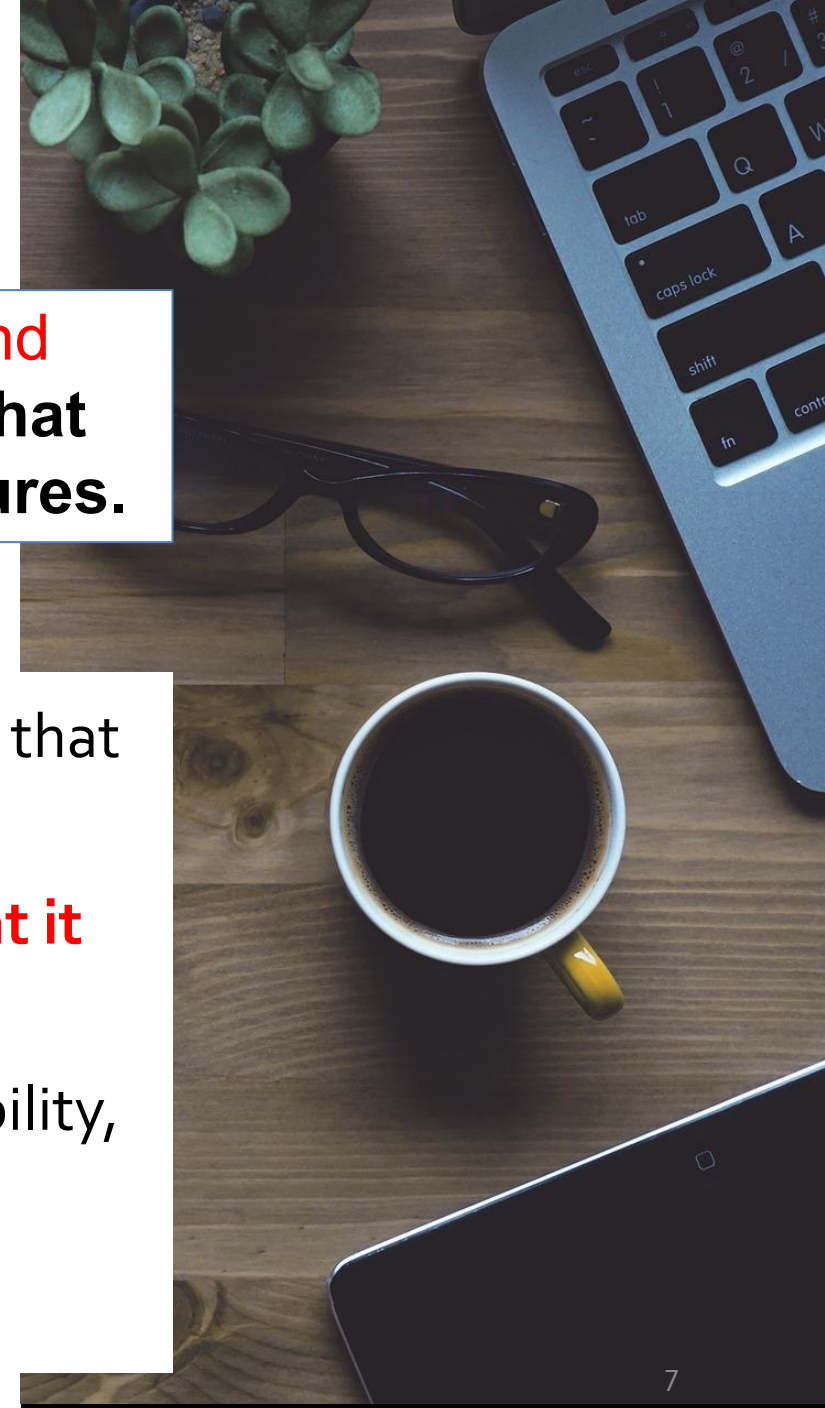
8.5 Safety

8.6 Security

8.2 Dependability

In simple terms, **dependability** refers to how reliable and consistent a system is. A dependable system is one that will work as expected, without any problems or failures.

- System dependability reflects the user's degree of trust in that system
- A dependable system **will operate as expected and that it will not fail in normal use.**
- Covers the related systems attributes of availability, reliability, safety and security.
 - These are all *inter-dependent*.



Dimensions of Dependability

8.2 Dependability

Dependability refers to how reliable and consistent a system is. A dependable system is one that will work as expected, without any problems or failures.

Dependability reflects the user's degree of trust in that system.

Dependability

Key **factors** that affect dependability are:

Availability

Reliability

Safety

Security

Resilience

The ability of a system to be **up and running** and able to deliver useful services to users when needed

The ability of a system to **correctly deliver services** as expected by users over a period of time.

The ability of a system to **prevent or mitigate** accidents, injuries, and other harmful events.

The ability of a system to **resist** accidental or deliberate attacks.

The ability of a system to **maintain continuity of its critical services** in the presence of disruptive events such as equipment failure and cyber-attacks, etc

Dimensions of **Dependability**

Availability, reliability, safety, security and resilience can also be describe as

Availability	The <u>probability</u> that the system will be up and running and able to deliver useful services to users.
Reliability	The <u>probability</u> that the system will correctly deliver services as expected by users.
Safety	A <u>judgment</u> of how likely it is that the system will cause damage to people or its environment.
Security	A <u>judgment</u> of how likely it is that the system can resist accidental or deliberate attacks.
Resilience	A <u>judgment</u> of how well a system can maintain the continuity of its critical services in the presence of disruptive events such as equipment failure and cyber-attacks.

Other Dependability Properties

8.2 Dependability

Repairability	Reflects the extent to which the system can be repaired in the event of a failure.
Maintainability	Reflects the extent to which the system can be adapted to new requirements.
Error tolerance	Reflects the extent to which user input errors can be avoided and tolerated.

Dependability Properties Inter-Dependencies

8.2 Dependability

- **Availability & reliability**: Safe system operation depends on the system being available and operating reliably.
- **Reliability & security**: A system may become unreliable because its data has been corrupted by an external attack.
- **Availability & security** : Denial of service (DoS) attacks on a system are intended to make it unavailable.
- **Reliability, safety & security**: If a system is infected with a virus, you cannot be confident in its reliability or safety.



The Costs of Dependability

8.2 Dependability

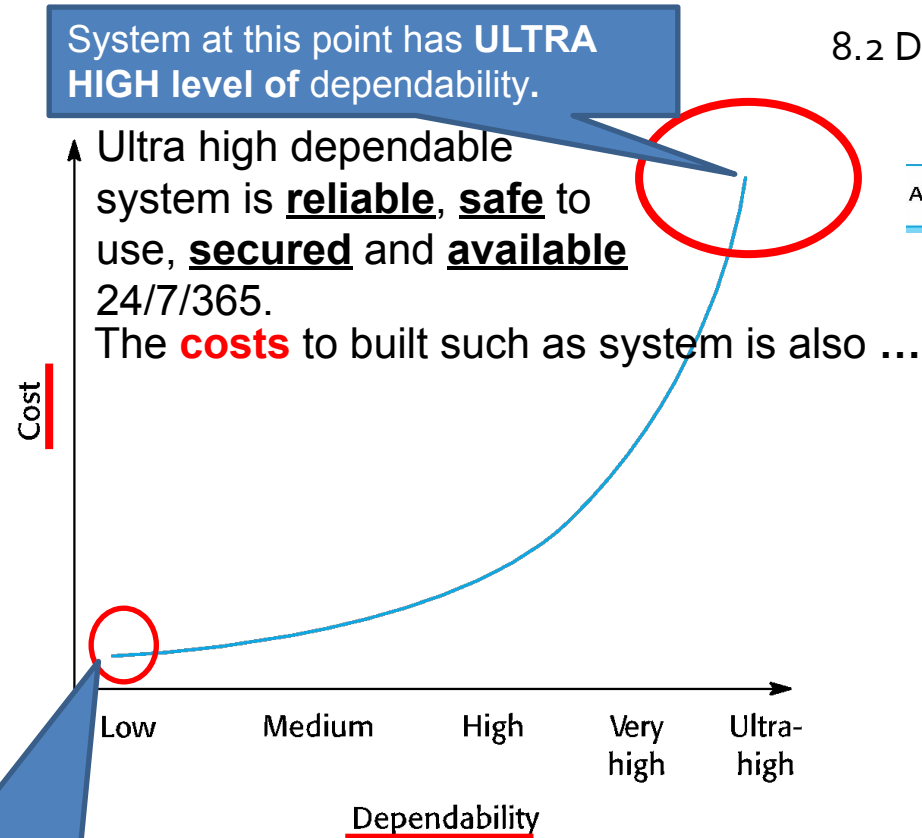
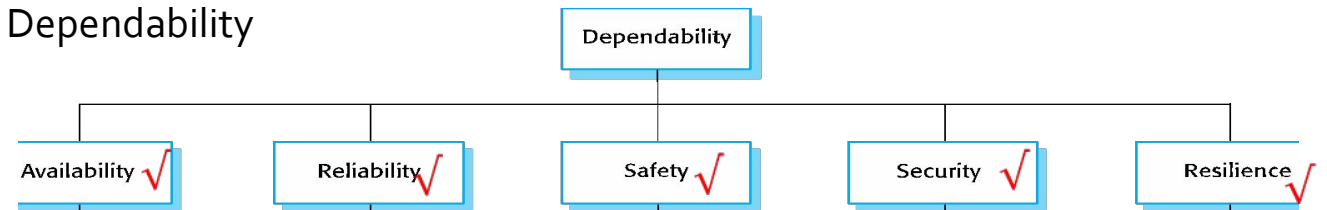


Fig: Cost/Dependability Curve



When the required level of dependability increased, the costs to development also increased (exponentially)

Reasons:

- The **use of more expensive development techniques and hardware** to achieve higher levels of dependability.
- The **increased testing and system validation** to convince the client and regulators that the required levels of dependability have been achieved.

...

Dependability level = **LOW**

If the dependability requirement is **LOW**, the **costs** to built such system is also **LOW**

8.3 Achieving Dependability

Q) How to develop dependable software/system?

- **Avoid introducing errors** into the system **during development** (because errors will cause failures)
- **Design Verification & Validation processes** (V&V processes) that are effective in finding **residual errors** in the system.
(Remaining bugs in sys even after performing thorough testing)
- **Design systems to be fault tolerant** so that it **can continue working when things go wrong.**
- **Design protection mechanisms** that **guard against external attacks** that can compromise the availability or security of the system

OTHER WAYS:

- **Configure the system correctly** for its operating environment.
- **Include system capabilities to recognize cyberattacks and to resist these attacks.**
- **Design recovery mechanisms** to help restore system after a failure/cyber attack.

Redundancy and diversity

8.3 Achieving Dependability

2 approaches :

- **Redundancy:** Keep more than one version of critical components so that a backup is available.
 - E.g., if availability is critical (e.g. e-commerce systems), companies **keep backup servers and switch to backup servers automatically if failure occurs.**
- **Diversity:** Provide the same functionality in different ways in different components so that they will not fail in the same way.
 - E.g., to provide resilience against external attacks, **have different servers using different operating systems (e.g. Windows Server & Linux)**
- Redundant and diverse components should be independent to avoid 'common-mode' failures
 - e.g., components implemented in different programming languages means that a compiler fault will not affect all of them.

A photograph of a wooden desk with a laptop, a pair of glasses, and a mouse. The laptop is silver and closed, with a pair of black-rimmed glasses resting on it. A white computer mouse is visible in the foreground. The background is a plain, light-colored wall.

Process Diversity and Redundancy

8.3 Achieving Dependability (con't)

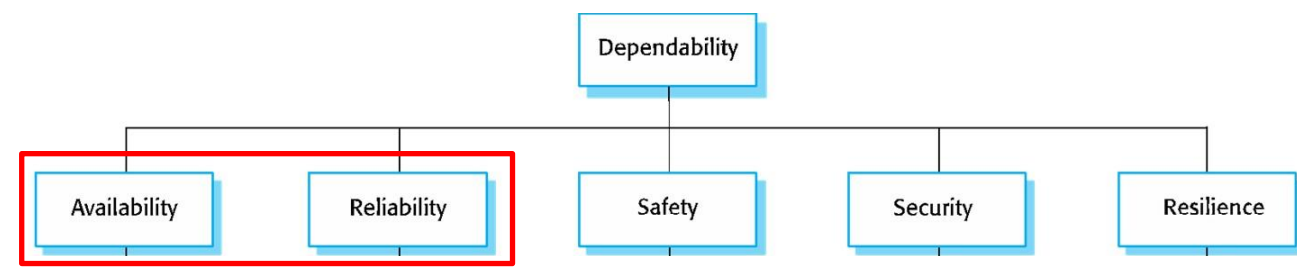
- Process activities such as **system validation, should not depend on a single approach, such as testing, to validate the system.** *(This means that do not' depends on "testing" alone to ensure system is free from errors. Something else need to be done too. E.g _____)*
- Redundant and diverse process activities are important especially for verification and validation.
- Multiple, different process activities that complement each other and allow for cross-checking help to avoid process errors, which may lead to errors in the software.

Process Diversity and 8.2 Achieving Dependability Redundancy

Challenges with redundancy & diversity

- Adding diversity and redundancy to a system increases the system complexity.
- This can increase the chances of error because of unanticipated interactions and dependencies between the redundant system components.
- Some engineers therefore advocate simplicity and extensive verification & validation as a more effective route to software dependability.

8.4 Availability and reliability

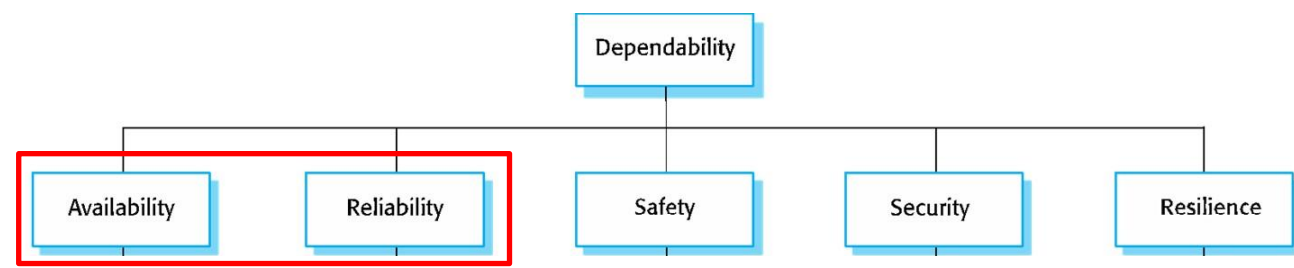


- **Availability** - The ability of a system at a point in time, to be operational (up & running) and able to deliver useful services to users when needed
- **Reliability** - The ability of a system to correctly deliver services as expected by users, over a period of time, in a given environment for a given purpose.

e.g: A banking system correctly transferred an amount of money from acct A to acct B every time.

- Availability & Reliability can be expressed **quantitatively**:
- E.g. System XYZ availability is 99.9%. This means that XYZ is operational 99.9% of the time over a given period
 - e.g1:** over a period of 1 month(30 days), 99.9% of availability means sys availability is 29.97days, amt of downtime = 0.03 days (43.2 mins)
 - e.g2:** over a period of 1 year(365 days), 99.9% of availability means sys availability is 364.64days amt of downtime = 0.36 days (8.64 hrs) (518 mins)

8.4 Availability and reliability



- **Availability** - The ability of a system at a point in time, to be operational (up & running) and able to deliver the requested services.
- **Reliability** - The ability of a system to correctly deliver services as expected by users, over a period of time, in a given environment for a given purpose.

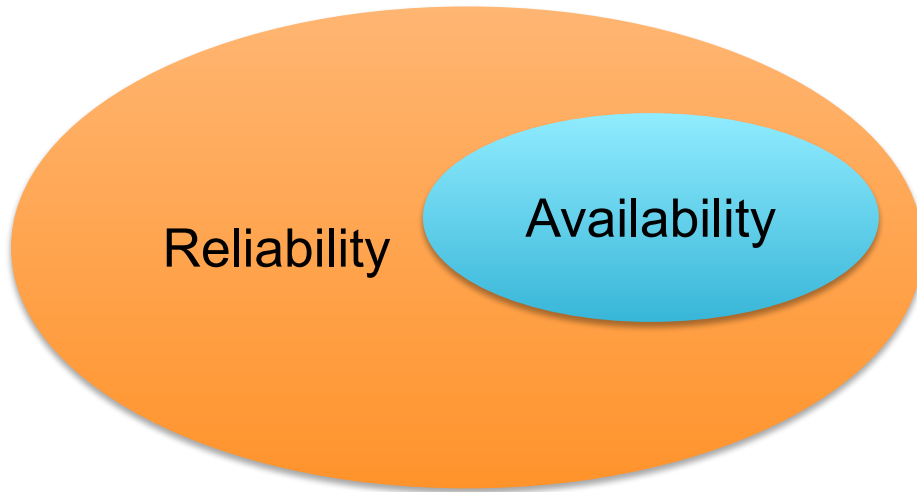
e.g: A banking system **correctly** transferred an amount of money from acct A to acct B every time.

This banking system is a **reliable** system as it meet the user's **expectation** every time.

- Availability & Reliability can be expressed **quantitatively**:
- For reliability, if on average, 2 inputs in every 1,000 cause failures, then the level reliability is **99.8%**. *because failure rate = 0.2% (2/1000 * 100%)*

Availability & Reliability can be expressed **quantitatively**:

Availability and Reliability

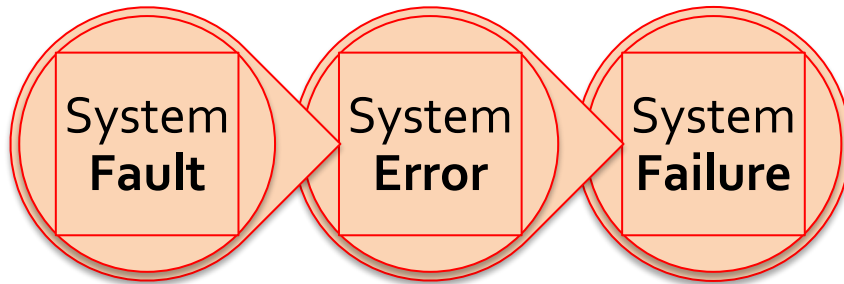


Availability and **Reliability** are closely related

- **If a system is unavailable then it is not reliable**
(e.g. an ATM machine that is often out of service is unreliable)
because it is not able to deliver services as expected by its users
- **Availability** **affect** **Reliability**

Faults and Failures

8.4 Availability & Reliability



- Failures are usually a result of system errors that are derived from faults in the system
- However, faults do not necessarily result in system errors
 - The erroneous system state resulting from the fault may be transient and 'corrected' before an error arises.
 - The faulty code may never be executed.
- Errors do not necessarily lead to system failures
 - The error can be corrected by built-in error detection and recovery
 - The failure can be protected against by built-in protection facilities. These may, for example, protect system resources from system errors

Faults and Failures

at some point in time, the system is not able to deliver a service as expected by its users



Reason:

The faulty code is not executed because of the way the SW is used
(each time the SW is used, not all codes in a program are executed)

Errors occur briefly.

(e.g. A variable may have an **incorrect value** caused by the execution of faulty code. **But, before this is accessed and caused a system failure, built-in error detection algorithm resets the variable to a valid value.**

E.g Human Error, System Fault, System Error and System Failure

8.4 Availability & Reliability

Term	Description
Human error	Human behavior that results in the introduction of faults into a system. <i>E.g.</i>, for hourly weather broadcast, the programmer introduces a logic error: he computes the time for the next transmission just by adding 1 hour to the current time. This works except when the transmission time is between 23.00 and midnight (midnight is 00.00 in the 24-hour clock).
System fault	A characteristic of a software system that can lead to a system error. <i>E.g.</i>, the fault is the inclusion of the code to add 1 hour to the time of the last transmission, without a check if the time is greater than or equal to 23.00.
System error	An erroneous system state that can lead to system behavior that is unexpected by system users. <i>E.g.</i>, the value of transmission time is set incorrectly (to 24.XX rather than 00.XX) when the faulty code is executed.
System failure	An event that occurs at some point in time when the system does not deliver a service as expected by its users. <i>E.g.</i>, no weather data is transmitted because the time is invalid.

Fault Management

8.4 Availability & Reliability

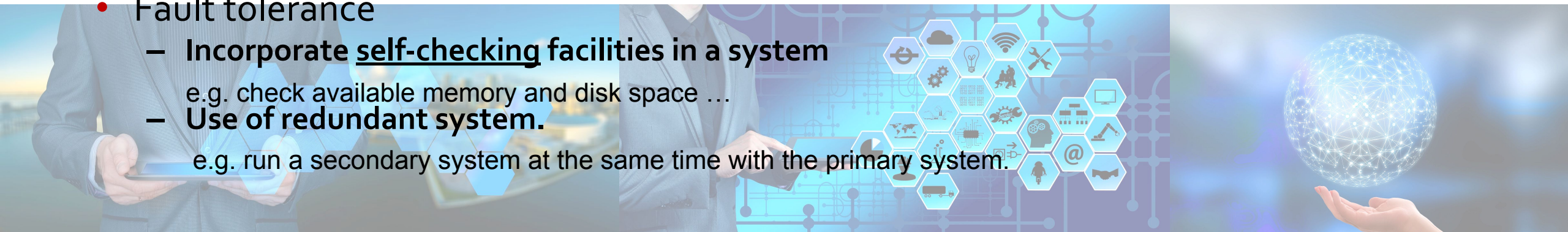
3 approaches to improve system reliability:

- Fault avoidance
 - Develop techniques to either minimize the possibilities of mistakes and/or trap mistakes before these result in system failure
 - e.g. avoid error-prone programming language constructs like pointers, use static analysis to detect program anomalies.
- Fault detection
 - Use verification and validation techniques to discover and remove faults in a system before it is deployed.
 - e.g. implement systematic system testing and debugging.
- Fault tolerance
 - Incorporate self-checking facilities in a system
e.g. check available memory and disk space ...
 - Use of redundant system.
e.g. run a secondary system at the same time with the primary system.

check program codes to find unusual or problematic patterns.

check whether the software **development process** is carry out correctly

check whether **developed system** is working as expected.



Availability Perception

8.4 Availability & Reliability

- Availability is usually expressed as a percentage of the time that the system is available to deliver services (e.g. 99.9%)
e.g. over a 30days period, system availability is 29.97days (amt of downtime = 0.03days = 43.2 mins)
- **Perception of availability** (*2 factors affect the perception of availability*)
 - (i) The *number of users affected by system failure*.
 - Loss of service in the middle of the night is less important for many systems than loss of service during peak usage periods.
 - (ii) *Duration of system unavailability*
 - The longer the outage, the more the disruption.
 - Several short outages are less likely to be disruptive than one long outage.

Remember: **Availability** affect **Reliability** (IF a system is unavailable then it is unreliable)

Specifying Reliability Requirements

8.4 Availability & Reliability

- Functional reliability requirements define system and software functions that avoid, detect or tolerate faults in the software and so ensure that these faults do not lead to system failure.
- Reliability is a measurable system attribute so non-functional reliability requirements may be specified quantitatively. These **define the number of failures that are acceptable during normal use of the system** **OR** the time in which the system must be available.



Reliability Metrics

8.4 Availability & Reliability

- System reliability is measured by counting the number of operational failures and, where appropriate, relating these to the demands made on the system and the time that the system has been operational.
- A long-term measurement programme is required to assess the reliability of critical systems.

- **Metrics**

- **Calculate the Rate Of Occurrence Of Failures (ROOOF) over a period of time**

(e.g. An e-commerce website experiences 5 failures (such as inability to process payments) from 6-11pm.
 $ROOOF = \text{no of failures} / \text{time of observation}$
 $= 5 / 5(6-11PM) = 1 \text{ failure per hour}$

- **Mean time to failure.**

(*average time(e.g. in years) a system or component is expected to operate before experiencing a failure*)

- **Availability**

(*measure uptime or downtime over a period of time e.g. per week/month/year*)



Rate of Fault Occurrence (ROCOF)

8.4 Availability & Reliability

- Reflects the rate of occurrence of failure in the system.
- ROCOF of 0.002 means 2 failures are likely in each 1000 operational time units e.g. 2 failures per 1000 hours of operation.
- Relevant for systems where the system has to process a large number of similar requests in a short time
 - Credit card processing system, airline booking system



Mean Time to Failure (MTTF)

8.4 Availability & Reliability

- The predicted elapsed time between inherent failures of the system during normal system operation.
- Relevant for systems with long transactions i.e. where system processing takes a long time.
- MTTF should be longer than expected transaction length.



Availability

8.4 Availability & Reliability

- Measure of the fraction of the time that the system is available for use.
- Takes repair and restart time into account
- Availability of 0.998 means software is available for 998 out of 1000 time units.
- **Important for non-stop, continuously running systems**
 - E.g telephone switching systems, railway signalling systems.
 - Can you give 1 example? _____



Availability

8.4 Availability & Reliability

Availability specification

Availability	Explanation
0.9	The system is available for 90% of the time. This means that, in a 24-hour period (1,440 minutes), the system will be unavailable for 144 minutes.
0.99	In a 24-hour period, the system is unavailable for 14.4 minutes.
0.999	In a 24-hour period, the system is unavailable for 84 seconds
0.9999	The system is unavailable for 8.4 seconds in a 24-hour period. Roughly, one minute per week.





System Dependability & Critical Systems

Table of Contents

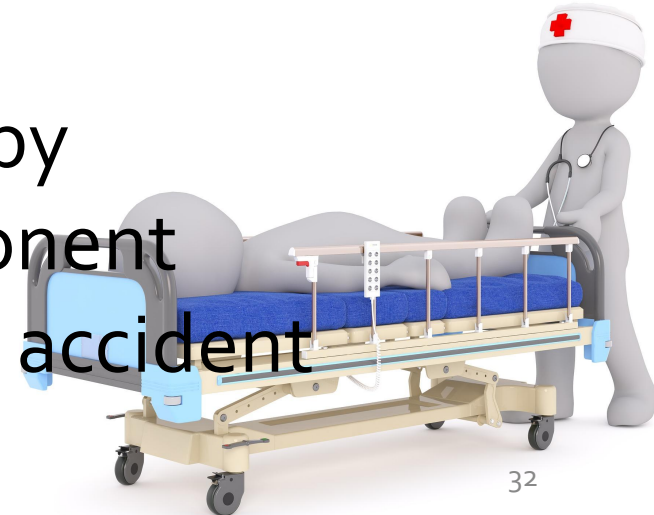
- 8.1 Critical Systems ✓
- 8.2 Dependability ✓
- 8.3 Achieving Dependability ✓
- 8.4 Availability & Reliability ✓
- 8.5 Safety**
- 8.6 Security

8.5 Safety

- **Safety** is the ability of a system to prevent or mitigate accidents, injuries and other harmful events.
- Software is extensively used for checking and monitoring safety-critical components in a system.

E.g.

All aircraft engine components are monitored by software looking for early indications of component failure. This software is safety-critical, if it fails, accident may happened.



Safety vs. Reliability

8.5 Safety

- **Safety** is the ability of a system to prevent or mitigate accidents, injuries and other harmful events.
- Software is extensively used for checking and monitoring safety-critical components in a system.
E.g.
All aircraft engine components are monitored by software looking for early indications of component failure. This software is safety-critical, if it fails, accident may happened.

Reliability is the ability of a system to correctly deliver services as expected by users over a period of time.

- **A reliable system can be unsafe.**
(i.e a system that correctly deliver services as expected by users can be unsafe!)

Why a reliable system can be unsafe:



E.g. a technician pressed a button that instructed the aircraft utility management software to raise the undercarriage. The software carried out the technician's instruction perfectly.

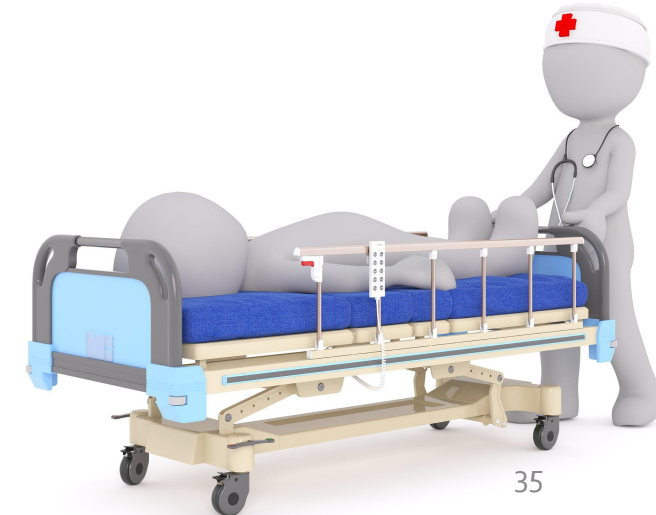
What will happen if the system carry out the instruction? _____

To ensure safety, the command is allow only when the plane is in the air.

Safety Critical Systems

8.5 Safety

- Systems where it is essential that system operation is always safe i.e. the system should never cause damage to people or the system's environment.
- Examples
 - Control and monitoring systems in aircraft
 - Process control systems in chemical manufacturing plant
 - Automobile control systems such as braking and engine management systems



Safety Criticality

8.5 Safety

Primary safety-critical systems

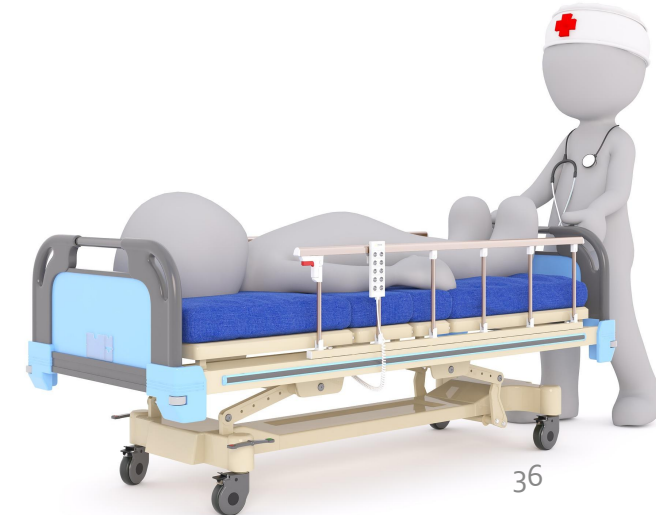
- Embedded software systems whose failure can cause the associated hardware to fail and directly threaten people.
- Example: the insulin pump control system.

Secondary safety-critical systems

- Systems whose failure results in faults in other systems, which can then have safety consequences.

Examples:

- The Patient Information System is safety-critical as failure may lead to inappropriate treatment being prescribed.
- Infrastructure control systems are also secondary safety-critical systems.



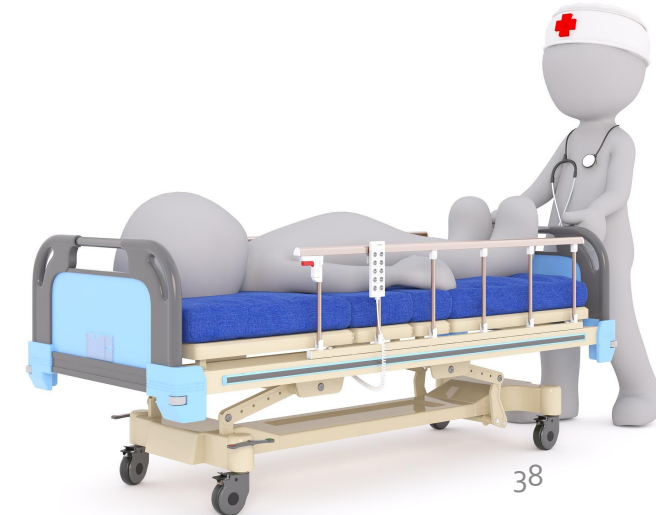
3 Ways to Achieve Safety For Safety-Critical System:

1. **Design system to avoid hazards.**
 - E.g. a microwave oven door must be fully closed before cooking is allowed. This is to avoid user from being exposed to microwave.
2. **Design system to detect hazards and removed it before they result in an accident.**
 - E.g. when excessive pressure is detected in a device or system, then automatically open a relief valve. This is to avoid explosion.
3. **Built-in protection features in a system to minimize damage**
 - E.g. includes an automatic fire extinguishers in an aircraft engine. If a fire occurs, it can be controlled before it poses a threat to the aircraft

Examples of Safety Requirements

8.5 Safety

- SR1:** The system shall not deliver a single dose of insulin that is greater than a specified maximum dose for a system user.
- SR2:** The system shall not deliver a daily cumulative dose of insulin that is greater than a specified maximum daily dose for a system user.
- SR3:** The system shall include a hardware diagnostic facility that shall be executed at least four times per hour.
- SR4:** The system shall include an exception handler for all of the exceptions that are identified in Table 3.
- SR5:** The audible alarm shall be sounded when any hardware or software anomaly is discovered and a diagnostic message, as defined in Table 4, shall be displayed.
- SR6:** In the event of an alarm, insulin delivery shall be suspended until the user has reset the system and cleared the alarm.



8.6 Security

- The ability of a system to **resist** accidental or deliberate attacks
- Security is an essential **pre-requisite** for availability, reliability and safety
 - ***Security** must come first before availability, reliability and safety*



Security Dimensions

8.6 Security

Impact of security breach/failure

System may become unavailable
e.g. ATM

Without/lack of security: criminal may steal all the cash in the machine

Result: The ATM service will become unavailable

Confidentiality

- **confidential information is exposed** to people who are not authorised to read or use that information How to protect confidential information? ____

Integrity

Information in a system may be damaged or corrupted making it unusable or unreliable.

How to prevent information in a system from being damaged or corrupted? ____



Security Levels

8.6 Security

Infrastructure security

concerned with maintaining the security of all systems and networks that provide an infrastructure and a set of shared services to the organization.

Application security

concerned with the security of individual application systems or related groups of systems.

Operational security

is concerned with the secure operation and use of the organization's systems.



Application vs. Infrastructure Security

8.6 Security

- Application security is a software engineering problem where the system is designed to resist attacks.
- Infrastructure security is a systems management problem where the infrastructure is configured to resist attacks.
- The focus of this chapter is application security rather than infrastructure security.



System Security Management

8.6 Security

User and permission management

Adding and removing users from the system and setting up appropriate permissions for users

Software deployment and maintenance

Installing application software and middleware and configuring these systems so that vulnerabilities are avoided.

Attack monitoring, detection and recovery

Monitoring the system for unauthorized access, design strategies for resisting attacks and develop backup and recovery strategies.



Operational Security

8.6 Security

- Primarily a human and social issue
- Concerned with ensuring the people do not take actions that may compromise system security
 - E.g. Tell others passwords, leave computers logged on
- Users sometimes take insecure actions to make it easier for them to do their jobs
- There is therefore a trade-off between system security and system effectiveness.



Security terminology

Term	Definition
Asset	Something of value which has to be protected; may be the software system itself or data used by that system.
Attack	An exploitation of a system's vulnerability. Generally, this is from outside the system and is a deliberate attempt to cause some damage.
Control	A protective measure that reduces a system's vulnerability. Encryption is an example of a control that reduces a vulnerability of a weak access control system
Exposure	Possible loss or harm to a computing system. This can be loss or damage to data, or can be a loss of time and effort if recovery is necessary after a security breach.
Threat	Circumstances that have potential to cause loss or harm. You can think of these as a system vulnerability that is subjected to an attack.
Vulnerability	A weakness in a computer-based system that may be exploited to cause loss or harm.

Examples of security terminology in a Patient Information System

Term	Example
Asset	The records of each patient that is receiving or has received treatment.
Exposure	Potential financial loss from future patients who do not seek treatment because they do not trust the clinic to maintain their data. Financial loss from legal action by a patient. Loss of reputation.
Vulnerability	A weak password system which makes it easy for users to set guessable passwords. User IDs that are the same as names.
Attack	An impersonation of an authorized user.
Threat	An unauthorized user will gain access to the system by guessing the credentials (login name and password) of an authorized user.
Control	A password checking system that disallows user passwords that are proper names or words that are normally included in a dictionary.

Security vs. Safety

8.6 Security

Safety	Security
Safety problems are accidental – the software is not operating in a hostile environment.	In security, you must assume that attackers have knowledge of system weaknesses.
When safety failures occur, you can look for the root cause or weakness that led to the failure.	When failure results from a deliberate attack, the attacker may conceal the cause of the failure.
Shutting down a system can avoid a safety-related failure.	Causing a shut down may be the aim of an attack.
Safety-related events are not generated from an intelligent adversary.	An attacker can probe defenses over time to discover weaknesses.

Types of Security Requirement

8.6 Security

- Identification requirements
- Authentication requirements
- Authorization requirements
- Immunity requirements
- Integrity requirements
- Intrusion detection requirements
- Non-repudiation requirements
- Privacy requirements
- Security auditing requirements
- System maintenance security requirements



Security Requirement

8.6 Security

Security requirements for Patient Information System

- Patient information shall be downloaded at the start of a clinic session to a secure area on the system client that is used by clinical staff.
- All patient information on the system client shall be encrypted.
- Patient information shall be uploaded to the database after a clinic session has finished and deleted from the client computer.
- A log on a separate computer from the database server must be maintained of all changes made to the system database.



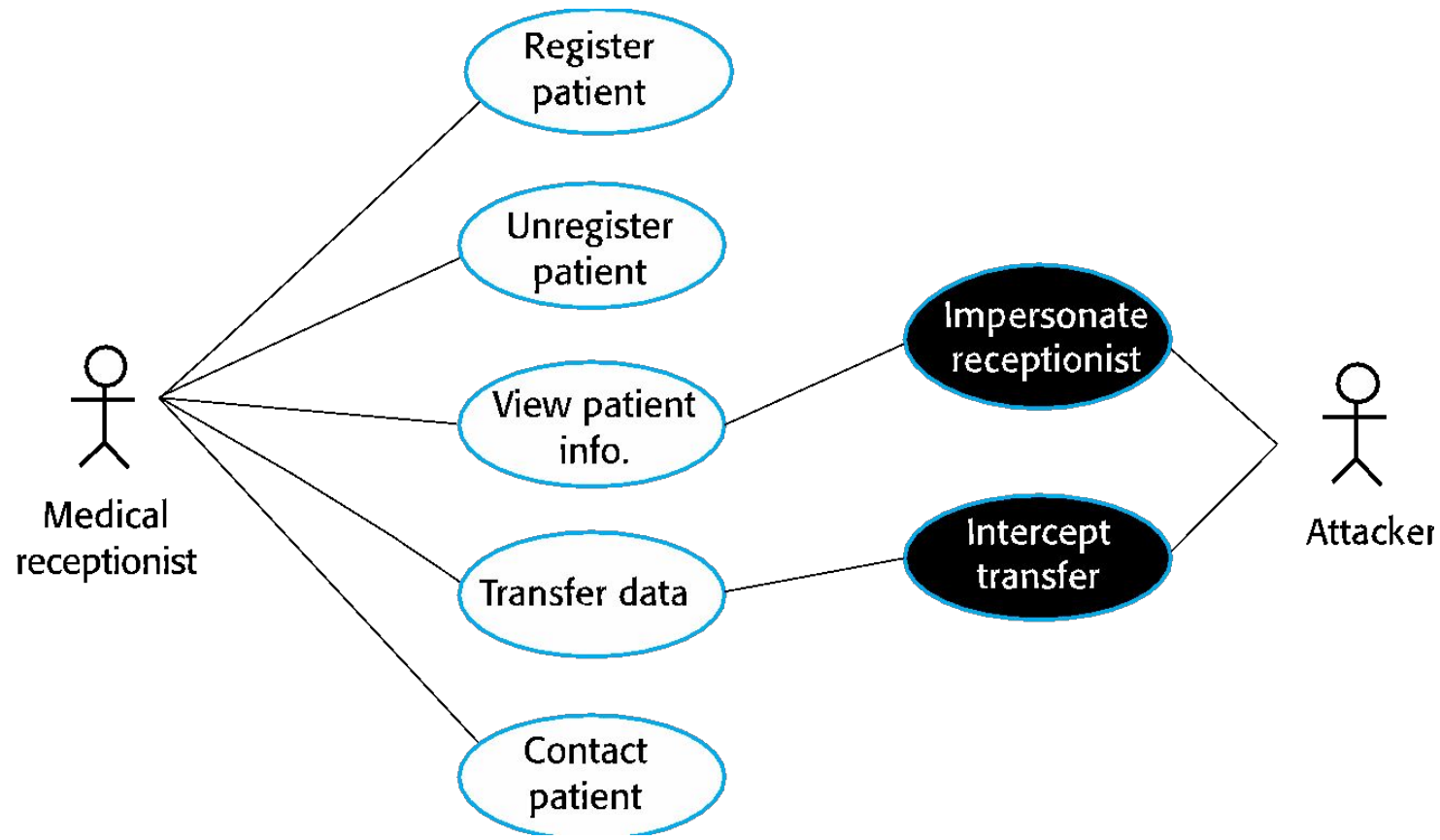
Misuse Cases

8.6 Security

- Misuse cases are instances of threats to a system
- Interception threats
 - Attacker gains access to an asset
- Interruption threats
 - Attacker makes part of a system unavailable
- Modification threats
 - A system asset is tampered with
- Fabrication threats
 - False information is added to a system



Misuse cases for Patient Information System



Use case – Transfer data

Patient IS: Transfer data

Actors	Medical receptionist, Patient records system (PRS)
Description	A receptionist may transfer data from the Patient IS to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
Data	Patient's personal information, treatment summary.
Stimulus	User command issued by medical receptionist.
Response	Confirmation that PRS has been updated.
Comments	The receptionist must have appropriate security permissions to access the patient information and the PRS.

Misuse case: Intercept transfer

Patient IS: Intercept transfer (Misuse case)	
Actors	Medical receptionist, Patient records system (PRS), Attacker
Description	A receptionist transfers data from his or her PC to the Patient IS on the server. An attacker intercepts the data transfer and takes a copy of that data.
Data (assets)	Patient's personal information, treatment summary
Attacks	A network monitor is added to the system and packets from the receptionist to the server are intercepted. A spoof server is set up between the receptionist and the database server so that receptionist believes they are interacting with the real system.
Mitigations	All networking equipment must be maintained in a locked room. Engineers accessing the equipment must be accredited. All data transfers between the client and server must be encrypted. Certificate-based client-server communication must be used
Requirements	All communications between the client and the server must use the Secure Socket Layer (SSL). The https protocol uses certificate based authentication and encryption.



Secure Systems Design

8.6 Security

- Security should be designed into a system – it is very difficult to make an insecure system secure after it has been designed or implemented
- Adding security features to a system to enhance its security affects other attributes of the system
 - Performance
 - Additional security checks slow down a system so its response time or throughput may be affected
 - Usability
 - Security measures may require users to remember information or require additional interactions to complete a transaction. This makes the system less usable and can frustrate system users.

Security Requirements

8.6 Security

- A password checker shall be made available and shall be run daily. Weak passwords shall be reported to system administrators.
- Access to the system shall only be allowed by approved client computers.
- All client computers shall have a single, approved web browser installed by system administrators.



How to Improve Security?

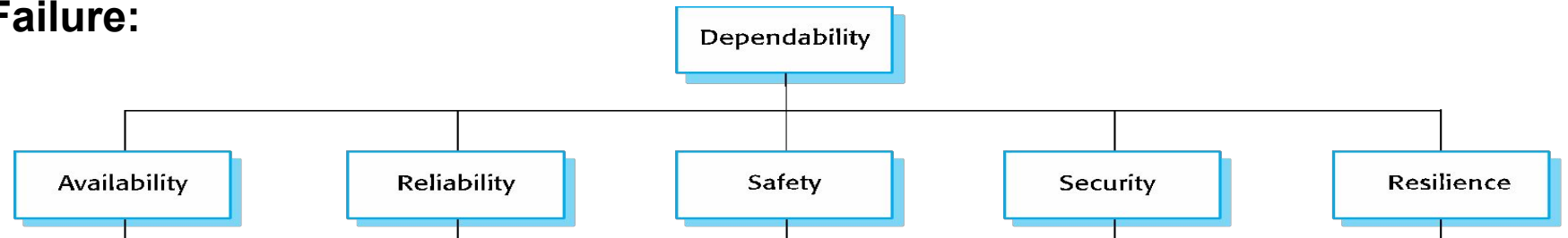
- **Avoid vulnerability :**
 - Design system so that vulnerabilities do not occur
 - **E.g. do not connect system to external public network** then there is no possibility of an attack from members of the public
(make a choice between convenience and security)
- **Implement attack detection and elimination:**
 - Design system with the ability to detect vulnerability and remove them before they result in an exposure
 - **E.g. use virus checkers to find and remove viruses before they infect a system**
- **Limit exposure:**
 - **Design system so that successful attack are minimised.**

Summary

8.1 Critical Systems

- **What is a critical system?**
- **3 types of critical systems**
 - Safety critical
 - Mission critical
 - Business critical
- **Causes of System Failure:**
- **Costs of Failure**

8.2 Dependability



Dependability is an essential attribute of **critical systems**

8.3 Achieving Dependability

8.4 Availability & Reliability

8.5 Safety

8.6 Security

Answer to the questions in this chap

8.6 Security

- Protect your document with a password
- Use antivirus software



the end